PHASE 1 PROJECT REPORT

# LinkUp

A Collaborative Digital Platform for Connected Learning and Teamwork

**Project Type** Web-Based Collaborative Digital Platform

**Current Phase** Phase 1 — Requirement Analysis, Research, Planning & Documentation
**Team** Sejal Naik · Sakshi · Siddhi · Harsh **Initial Target**

**Users** Students and Teachers

**Document Version** 1.0

**Date** October 2026

# Table of Contents


1. Project Overview 3
2. Problem Statement 3
3. Project Vision & Proposed Solution 3
4. Requirement Analysis 3
5. Research 4
6. Technology Stack 4
7. Proposed Technical Flows 5
8. System Architecture 6
9. Planning 7
10. Future Scope 7
11. Conclusion 7

# 1. Project Overview

LinkUp is a proposed web-based collaborative workspace that brings communication, learning, practical work, collaboration and resource sharing into one connected platform. The initial use case is teacher–student collaboration; the architecture is planned to later support colleges, educational institutes, companies, developers, professional teams and organizations. LinkUp is not only an online classroom.

This report covers Phase 1 only. Nothing is claimed as implemented; development, testing and deployment are future work, not separate phases.

# 2. Problem Statement

Users depend on many disconnected tools for meetings, audio/video, chat, screen sharing, files, coding, code execution, whiteboards, assignments, exams, polls, attendance, announcements and recordings. This fragments the workflow: users switch between applications, and teachers/admins manage permissions across different tools. LinkUp proposes one workspace where these activities are accessible together.

# 3. Project Vision & Proposed Solution

**Vision:** Bring multiple collaboration activities into one workspace with centralized access and permission control.

**Central requirement:** *Users collaborate in one workspace while the person responsible for the session controls what each participant may access or do.*

| Module | Scope |
|---|---|
| Communication | Audio, video, chat, announcements, notifications |
| Collaboration | Screen sharing, whiteboard, shared coding, personal and teacher compilers |
| Access Control | Per-user permissions: mic, camera, screen sharing, whiteboard, code, files |
| Resources | File upload, sharing, organization, session recordings |
| Learning / Activity | Assignments, exams, polls |
| Tracking | Attendance, participation |
| Workspace | Dashboard, classes/rooms, user management |

LinkUp is not presented as a unique invention. Existing platforms provide many individual capabilities; this project studies them and evaluates how relevant capabilities can be combined into a more connected workflow.

# 4. Requirement Analysis

## 4.1 Functional Requirements

| ID | Area | Requirement |
|---|---|---|
| FR-1 | User Management | Registration/login, authentication, roles, profile |
| FR-2 | Rooms / Classes | Create, manage and join rooms; manage participants |
| FR-3 | Live Collaboration | Audio, video, screen sharing, real-time communication |
| FR-4 | Permissions | Individual, teacher/admin-controlled: microphone, camera, screen sharing, whiteboard, code, files |
| FR-5 | Collaboration Tools | Whiteboard, shared code editor, code execution, personal compiler |
| FR-6 | Academic Tools | Assignments, exams, polls, attendance |
| FR-7 | Resources | Files, recordings, announcements, notifications |
| FR-8 | Dashboard | Central access to rooms, activities, files, attendance, notifications |

### 4.2 Non-Functional Requirements

| ID | Category | Requirement |
|---|---|---|
| NFR-1 | Security | Authentication, role-based access, permission control, protected data |
| NFR-2 | Performance | Responsive real-time communication, fast interaction, efficient data handling |
| NFR-3 | Usability | Simple interface, easy navigation, clear controls |
| NFR-4 | Scalability | Support more users and more organizations/use cases |
| NFR-5 | Reliability | Stable communication, data consistency, handling of connection/disconnection |

# 5. Research

## 5.1 Why Research is Required

Research is done before implementation to understand existing solutions and how they solve similar problems; identify their strengths, limitations, gaps and opportunities; avoid reinventing existing solutions; compare tools for fit, integration, performance, scalability, security, documentation, developer support, free-tier availability and cost; and select appropriate technologies.

## 5.2 Existing Product Research

| Existing Solution | Strength | Limitation / Gap | LinkUp Opportunity |
|---|---|---|---|
| **Microsoft Teams** | Meetings, audio/video, screen sharing, whiteboard, chat, files, assignments, recordings | Broad general-purpose suite; coding and code execution are not its core focus | Add coding and execution under one permission model |
| **Google Classroom** | Classroom management, assignments, resources, communication | Live sessions and other tools generally rely on connected products | Combine classroom activities with live collaboration |
| **Moodle** | Courses, assignments, quizzes, learning activities | LMS-centred; real-time collaboration usually via plugins/integrations | Pair activity management with built-in real-time tools |
| **Replit** | Collaborative coding and code execution | Coding-focused; not primarily built around class management or exams | Offer coding as one module among many |
| **Miro** | Collaborative whiteboarding | Whiteboard-focused; not a full meeting or activity platform | Embed whiteboarding in the shared workspace |

These products are studied, not criticised. Observations are general and should be re-verified against current official documentation.

## 5.3 Research Findings / Gaps

Existing platforms already provide many of the individual capabilities required by LinkUp. However, these capabilities are distributed across different products or designed around particular use cases. LinkUp explores bringing communication, controlled collaboration, coding, whiteboarding, resource management and activity management into a unified workspace with a common permission model.

## 5.4 Technology & Tool Research

Each tool was assessed on: fit for the requirement, integration with React/Node.js/Supabase, performance and scalability, security, documentation and support, and cost/licensing (free tier or open source preferred). Items to verify before implementation: LiveKit free-tier limits, tldraw licence terms, and Piston public API availability versus self-hosting.

# 6. Technology Stack

Free-tier status is stated generally; exact pricing and limits must be verified from official documentation.

| Layer | Technology | Purpose | Free Tier / Status |
|---|---|---|---|
| Frontend | React + Vite | Web interface, component-based UI | Open source |
| UI | Tailwind CSS | Responsive, consistent styling | Open source |
| Backend | Node.js + Express | APIs, server logic, auth-related operations | Open source |
| Real-time | Socket.IO | Chat, attendance events, permission updates, sync | Open source |
| Audio/Video | WebRTC | Audio, video, screen sharing, rooms, recording | Free/developer usage; limits apply |
| Whiteboard | tldraw | Collaborative drawing | Free for eligible projects; check licence |
| Code Editor | Monaco Editor | Editing, shared workspace, syntax highlighting | Open source |
| Code Execution | Piston / self-hosted Piston | Run code, return output/errors | Self-hosting as open-source option |
| Auth | Supabase Auth | Login and sessions | Free tier |
| Database | Supabase PostgreSQL | Users, rooms, permissions, attendance, assignments, exams, polls, messages, recording metadata | Free tier |
| Storage | Supabase Storage | Files, resources, recordings | Free tier; recordings depend on usage |
| Tools | Git + GitHub, VS Code, Figma, draw.io, Google Docs/Word, Google Drive | Version control, development, design, diagrams, documentation, team resources | Free tier / free |

**Piston:** self-hosting can be considered for free/open-source execution. The public API's availability and limits must be verified; it is not assumed to be permanently free or unlimited.

# 7. Proposed Technical Flows

These are planned designs, not implemented work. Each line shows the flow; the note after "Build" says how it would be built.

**Authentication** — User → Supabase Auth Login → Frontend → Session / User ID → Backend → Verify User → Dashboard

*Build:* login page calls Supabase Auth; Express middleware verifies the token on every request.

**Video & Audio** — User → Frontend → Socket.IO Signaling → Peer Connection → Audio / Video Attendance — Join → Socket join → Backend → INSERT attendance(user_id, room_id, joined_at) → Supabase | Leave → Socket leave → Backend → UPDATE attendance SET left_at → Supabase

*Build:* open records are reused on reconnect.

**Permission Control** — Admin/Host → Select User → Set Permission → Frontend → Backend → Database → Socket Event → Enforcement

*Build:* Admin updates permissions → NestJS checks Admin role → DB updates → Socket.IO broadcasts event → Target user's browser disables WebRTC media tracks & locks UI controls → NestJS RBAC guards block any unauthorized server payloads (403 Forbidden).

**Chat** — User → Type Message → Frontend → Socket message → Backend → Store Message → Supabase → Broadcast → Other Users

**Screen Sharing** — User → Request Screen Share → Permission Check → Socket.IO Signaling → Peer Connection → Screen Stream → Room Participants

**Build:** Frontend requests screen media via `navigator.mediaDevices.getDisplayMedia()`; backend verifies screen-sharing permission via NestJS/Socket.IO guards; frontend renegotiates WebRTC peer connection by adding screen track; WebRTC streams screen directly to participants while Socket.IO broadcasts stream state

**Whiteboard** — User → Open Whiteboard → tldraw → Draw / Write → Collaboration Event → Socket → Other Users → Updated Whiteboard

*Build:* only users with whiteboard permission can edit; others view.

**Collaborative Code Editor** — User → Open Code Workspace → Monaco Editor → Edit Code → Socket → Sync Changes → Other Users

*Build:* only permitted users edit, one at a time under teacher control, to avoid conflicting edits.

**Code Execution** — User → Write Code → Click "Run" → Frontend → POST /execute → Backend → Piston → Output / Error → Frontend

*Build:* backend checks permission and limits code length/run time; Piston calls via POST /api/v2/execute. A personal compiler uses the same flow in a private editor.

**File Management** — User → Select File → Frontend → Upload Request → Backend → Supabase Storage → File URL / Metadata → Supabase Database → Authorized Users

**Assignments** — Teacher → Create Assignment → Backend → Supabase → Students Receive → Student Submits → Backend → Store Submission

**Exams / Quizzes** — Teacher → Create Exam → Questions + Options → Supabase → Student Opens Exam → Submit Answers → Backend → Evaluate / Store Result

*Build:* scoring on the server; correct answers are not sent to the browser beforehand.

**Polls** — Teacher → Create Poll → Socket / Backend → Students Receive Poll → Student Votes → Backend → Store Response → Live Results

**Announcements** — Teacher → Create Announcement → Backend → Supabase → Socket / Notification → Students → View

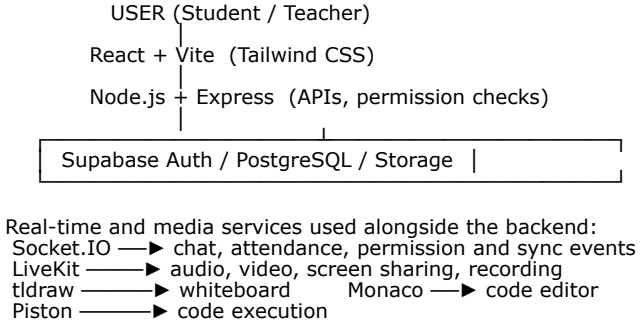
**Recording** — Teacher starts meeting → LiveKit Room → Recording Enabled → Recording File → Storage → Metadata → Supabase Database → Authorized Users

*Build:* LiveKit Egress writes the file to storage (verify Supabase's S3-compatible setup in official docs); a recordings table holds title, room, date, size and path.

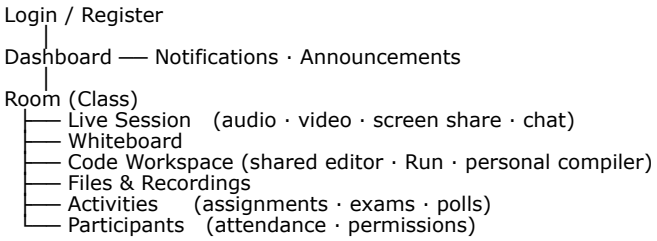
**Dashboard** — User Login → Supabase Auth → Frontend Dashboard → Request User Data → Backend → Supabase → Classes / Meetings / Assignments / Attendance / Files / Notifications → Dashboard

**Recording storage management:** a one-hour lecture is roughly 0.5–1 GB at 720p (estimate). Plan: one combined video, 720p, a maximum length, a per-teacher quota, auto-delete after a set period (e.g. 30–60 days) with a "keep" option, and storage usage shown on the dashboard. All data, including lecture records, stays in Supabase PostgreSQL; video files stay in Supabase Storage.

# 8. System Architecture




**Basic app structure (planned screens):**




The frontend is the single entry point. Express verifies users and permissions and talks to Supabase; Socket.IO carries real-time events; LiveKit handles media; tldraw, Monaco and Piston provide the whiteboard, editor and execution.

# 9. Planning

**Product planning:** build the modules in Section 3 using the stack in Section 6, in the group order below.

**Permission model:** the teacher/admin controls each participant's access per room.

| Permission | Controls |
|---|---|
| Microphone / Camera | Speaking and sharing video |
| Screen Sharing | Presenting a screen |
| Whiteboard | Access to / drawing on the whiteboard |
| Code Workspace | Access to the shared code workspace |
| Files | Access to shared files |

Permissions are stored per user per room in PostgreSQL, checked by the backend and updated live through Socket.IO.

**Development environment:** VS Code, a GitHub repository (client and server folders) and local development; Figma for design; draw.io for diagrams; Google Docs/Word for documentation. Deployment is future planning only.

**Technology selection:** tools were chosen using the criteria in Section 5.4, preferring free-tier or open-source options with good documentation.

**Feature prioritization** (planned implementation priority, not project phases):

| Priority Group | Features |
|---|---|
| Core Foundation | Authentication, roles, dashboard, rooms |
| Core Collaboration | Audio/video, screen sharing, chat, permissions |
| Interactive Workspace | Whiteboard, code editor, code execution, personal compiler |
| Management | Files, attendance, assignments, polls, exams |
| Supporting Features | Recordings, announcements, notifications |

# 10. Future Scope

10.1 Advanced Collaboration Features

Future versions could introduce:

- More advanced collaborative whiteboard tools, shapes, templates, and drawing controls.
- Improved real-time document editing.
- Collaborative code editing with support for more programming languages.
- Enhanced screen sharing and meeting controls.
- Shared workspaces for teams and projects.
- Version history and recovery for collaborative work.
- Better support for large groups and multiple simultaneous sessions.

10.2 Future Scope for Developers

LinkUp could become a shared workspace for software development teams:

- Advanced collaborative code editing.

- Support for additional programming languages and compilers.
- Code version history and change comparison.
- Git and GitHub integration.
- Shared debugging and code review workflows.
- Project task tracking and issue management.
- Team documentation and technical knowledge sharing.
- Integration with development tools and APIs.
- Collaborative project demonstrations and technical discussions.

10.3 Future Scope for Colleges and Educational Institutions

Future versions could support larger academic environments through:

- Institution and department management.
- Multiple classes, courses, and academic groups.
- Centralized course materials and resources.
- Advanced attendance reports and analytics.
- Assignment and assessment workflows.
- Learning progress dashboards.
- Integration with existing learning management systems (LMS).
- Administrative reporting and role-based access.
- Support for online workshops and academic events.

10.4 Future Scope for Businesses and Organizations

LinkUp could expand into a collaboration environment for companies and organizations:

- Organization-specific workspaces.
- Team and department management.
- Meeting notes and action-item tracking.
- Project planning and task assignment.
- Shared documents and resources.
- Team progress dashboards.
- Integration with calendars and productivity applications.
- Organization-level administration and access policies.
- Support for remote and hybrid teams.

10.5 Future Scope for Researchers

Researchers could use LinkUp to collaborate on academic, scientific, and technical work:

- Collaborative research discussions.
- Shared notes and reference materials.
- Collaborative analysis and code workspaces.
- Research project organization.
- Secure sharing of approved documents and datasets.
- Collaboration across institutions and research teams.
- Integration with suitable research and data-analysis tools.

10.6 AI-Powered Assistance

An AI assistant is a potential future enhancement, not a feature included in the current planned scope:

- Answering questions about permitted project or learning materials.
- Summarizing discussions and shared documents.
- Generating meeting notes and action items.
- Helping users understand code and technical concepts.
- Suggesting relevant learning or project resources.
- Supporting brainstorming and problem-solving.

10.7 Session Recording and Playback

Session recording is also a future-scope feature:

- Recording authorized meetings or classes.
- Providing playback for permitted users.
- Managing access to recordings.
- Applying configurable retention and deletion policies.
- Supporting captions or searchable transcripts, if approved.
- Providing storage-management controls.

# 11. Conclusion

Phase 1 of **LinkUp** successfully completes the initial analysis, research, and technical planning for a unified collaborative platform. It solves the problem of using multiple separate apps by bringing live video meetings, chat, whiteboards, collaborative coding, and assignment management together under one controlled system. By testing a technology stack built with React, Node.js, Express, Socket.IO, WebRTC/LiveKit, and Supabase, Phase 1 confirms that LinkUp is technically feasible, scalable, and ready for future development into schools, colleges, and development teams.

workspace with a common permission model, starting with teacher–student collaboration. All features and flows are planned designs; none is claimed as implemented.